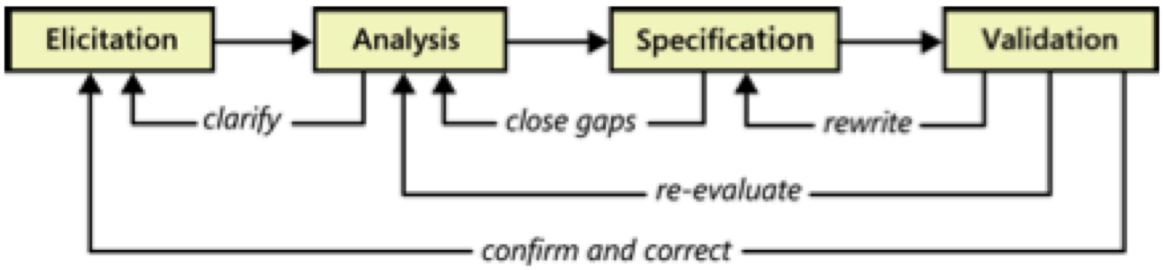
Webové aplikace - vývoj webu

Analýza

Analýza a specifikace softwarových požadavků (Requirements Engineering)

- Requirements Engineering je klíčová etapa vývoje softwaru, ve které se určuje co má systém dělat a jaké má mít vlastnosti a omezení
- Kvalitně zjištěné požadavky jsou základem úspěšného projektu - Naopak opomenuté nejednoznačné požadavky jsou častou příčinou selhání softwarových projektů
- Proces se skládá ze 4 hlavních fází, které na sebe navazují, ale nejsou striktně lineární - často se iterativně vracíme zpět a požadavky upřesňujeme



Iterativní cyklus analýzy a specifikace požadavků se zpětnými vazbami

- **Elicitace požadavků (zúčastněné strany)**
  - Elicitace požadavků je proces získávání informací o požadavcích od všech zúčastněných stran (**stakeholderů**)
  - Cílem je zjistit **potřeby** a představy uživatelů a zadavatelů o budoucím systému
  - **Na začátku je důležité identifikovat všechny zúčastněné strany:**
    - Koncoví uživatele
    - Administrátoři
    - Management
    - ....
  - *!!!!!! Opomenutí některé skupiny vede k tomu, že její požadavky zůstanou neobjeveny - a opraví se to pozdě a draze*

- **Techniky elicitace:**
  - Rozhovory s jednotlivými uživateli či experty
  - Skupinové workshopy a brainstorming
  - Dotazníky
  - Pozorování práce uživatelů (study tours)
  - Analýza stávající dokumentace a procesů
  - Tvorba prototypů
- V praxi se často kombinuje více přístupů, aby se získal co nejúplnější obraz požadavku
- Komunikace probíhá přirozeným jazykem, který je nejednoznačný. Uživatelé navíc často nedokážou své potřeby jasně formulovat
  - Úkolem analytika je aktivně klást doplňující otázky a nevnášet nepodložené předpoklady
- **Výstup elicitace**
  - **Předběžný seznam požadavků** (seznam funkcí, user stories, konceptuální modely)

- **Analýza požadavků**
  - Zjištěné požadavky se podrobně zkoumají a upřesňují
  - **Cílem této fáze je:**
    - Odstranit nejasnosti
    - Odhalit konflikty a duplicity
    - Zhodnotit proveditelnost
    - Stanovit priority
  - V této fázi může také dojít k vyjednávání o rozsahu požadavků - například pokud jsou některé požadavky nereálné vzhledem k omezením (časovým, rozpočtovým, technickým)
  - Analýza často zahrnuje tvorbu různých modelů a nástrojů, které pomáhají lépe pochopit a ověřit jejich úplnost
    - **Use Case diagramy** (UML) - kdo co v systému může dělat
    - **Náčrtky uživatelského rozhraní**
    - **Procesní diagramy** - tok činností
    - **Datové modely** (ER diagramy) - entity a jejich vztahy
  - Modely pomáhají odhalit mezery, které by v čistém textu nebyly vidět
  - Často se také stanovuje priorita jednotlivých požadavků (např. metoda **MoSCoW**), což pomůže při rozhodování, které požadavky mají přednost v případě omezených zdrojů
    - **MoSCoW** - Must have, Should have, Could have, Won't have
  - **Kontrola kvality požadavků:**
    - Každý požadavek bychom měli také zkontrolovat zda je:
      - **Srozumitelný** - nejlépe i pro jiné než autora
      - **Jednoznačný** - nepřipouští více výkladů
      - **Konzistentní** - neodporuje jiným požadavkům
      - **Ověřitelný** - dá se otestovat, zda je splněn

- *!!!!!! POZOR na záměnu požadavků a návrhu! V analýze se soustřeďte na co systém musí dělat, nikoli jak to bude implementováno. Formulace „databáze bude MySQL“ do požadavků nepatří.*
- **Výstup analýzy:**
  - Upřesněná, konsolidovaná sada požadavků s vyřešenými rozpory a stanovenými prioritami

- **Specifikace požadavků (SRS)**
  - **SRS** - Software Requirements Specification je formální dokument obsahující kompletní jednoznačný popis všech požadavků na systém
  - Slouží jako smlouva mezi zadavatelem a vývojovým týmem
  - **Struktura SRS dokumentu:**
    - Obsahuje úvod s kontextem a cíli systému, slovníček pojmů (definice důležitých termínů, aby nedocházelo k nejasnostem), a hlavně samotné požadavky (funkční a nefunkční požadavky)
  - **Funkční požadavky popisují co systém dělá - konkrétní funkce a chování**
    - Formulace by měla být jasná, měřitelná, s unikátním ID
    - **Příklad funkčního požadavku:**

      FR1: Obnovení hesla — Systém umožní registrovanému uživateli obnovit zapomenuté heslo pomocí jednorázového odkazu zaslaného na jeho e-mailovou adresu.

      FR2: Historie objednávek — Uživatel si může zobrazit přehled všech svých minulých objednávek včetně detailů položek a data nákupu.

  - **Nefunkční požadavky popisují jak systém funguje**
    - **Hlavní kategorie:**
      - Výkon
      - Dostupnost
      - Bezpečnost
      - Použitelnost apod.
    - **Příklad nefunkčního požadavku:**

      NFR1: Dostupnost systému – Webová aplikace musí být dostupná pro uživatele minimálně 99 % času v průběhu kalendářního měsíce.

- **Pravidla psaní specifikace:**
  - **správně**:
    - Jeden požadavek = jedna věta / jedna funkcionalita
    - Unikátní ID (FR1, FR2, NFR1)
    - Konkrétní měřitelné hodnoty („do 2 sekund“, ne „rychle“)
    - Konzistentní terminologie ze slovníčku
    - Akceptační kritéria
  - **Špatně**:
    - *„Systém bude uživatelsky přívětivý“*
    - *„Aplikace poběží rychle na každém počítači“*
- **Výstup specifikace:**
  - Kompletní **SRS dokument**, podle kterého se dá systém navrhnout, implementovat a otestovat

Validace požadavků

- Validace je ověření, zda sepsané a analyzované požadavky skutečně odpovídají potřebám zadavatele a uživatelů
- Je to závěrečná kontrola před návrhem a implementací
- Nalezené chyby a nedostatky v požadavcích a jejich oprava je nejméně nákladná
- Validace typicky probíhá formou revize specifikace společně se zadavateli a vybranými koncovými uživateli
- **Techniky validace:**
  - **Revize specifikace s klientem a uživateli** - projít dokument bod po bodu
  - **Walkthrough scénáře** - krok za krokem projít klíčový use case a ověřit, že požadavky pokrývají všechny kroky
  - **Prototyp** - uživatelé si konceptuálně vyzkouší, zda funkce splňují jejich očekávání
  - **Formální schválení (signoff)** - podepsané odsouhlasení klientem

- **Výstup validace:**
  - Buď schválená specifikace, nebo seznam úprav, které je potřeba zpracovat

- **Iterace a zpětné vazby (Proces není lineární):**
  - **Validace -> Specifikace**: Klient upřesní formulaci, přepíšeme dokument
  - **Specifikace -> Analýza**: Při psaní specifikace zjistíme, že analýza je neúplná
  - **Analýza -> Elicitace**: narazíme na rozpor, potřebujeme se zeptat uživatelů
  - **Validace -> Analýza**: Přehodnotíme priority nebo rozsah
  - **Validace -> Elicitace**: Objeví se úplně nový požadavek -> návrat na začátek

- **Proč je to tak důležité?:**
  - **Chyby v požadavcích jsou nejdražší** - čím později se odhalí, tím dražší oprava
  - **Specifikace je smlouva mezi klientem a vývojáři** - chrání obě strany
  - Kvalita softwaru stojí na kvalitě požadavků
  - Komunikace je klíčová

Návrh

- Na základě dokumentu se rozhoduje, jak bude systém vypadat a jak bude technicky fungovat. Odpovídá na otázku „JAK?“
- **UX a UI design (jak to bude vypadat a používat)**
  - **UX design (User Experience)**
    - **UX** = řeší, jak se web používá
    - **Nástroje UX designu:**
      - **User flow**
        - Diagram znázorňující cestu uživatele - od vstupu po dokončení cíle
      - **Wireframy**
        - Jednoduché náčrty rozložení stránky
  - **UI design (User Interface)**
    - Řeší jak web vypadá
  - **Důležité principy:**
    - **Responzivní design** - web musí fungovat na všech zařízeních
    - **Přístupnost** - web by měl být použitelný i pro uživatele s postižením
- **Rendering strategie (kde a kdy se skládá HTML)**
  - **SSR** (Server-Side Rendering)
  - **CSR** (Client-Side Rendering)
- **Technický návrh**
  - Z jakých částí bude systém postavený a jak spolu budou komunikovat
  - **Client-Server architektura:**
    - **Frontend (klient) - to, co běží v prohlížeči uživatele**
      - HTML
      - CSS frameworky: Boorstrap, Tailwind
      - JS frameworky: React, Vue.js, Angular, Svelte

- **Backend (server) - serverová logika, zpracování dat**
  - PHP (laravel, Symfony)
  - JavaScript/Node.js (Express, Next.js, NestJS
  - Python
  - Java
  - C#
  - Ruby
  - Go, Rust
- **Databáze**
  - Relační (MySQL, PostgreSQL, MariaDB, SQLite ….)
  - Nerelační (MongoDB, Redis …)
  - **Návrh databáze**
    - Jedna z nejdůležitějších částí
- **Návrh API**
  - Pokud frontend a backend komunikuji odděleně, definuje se struktura endpointů (REST API)

Implementace

- Tady píšeme kód
- **Verzování kódu (Git)**
  - **Proč**?
    - Historie změn, týmová spolupráce, větvení
  - **Platformy**: GitHub, GitLab
- **Code review**
  - Než se kód dostane do hlavní větve, projde ho někdo jiný z týmu
- **Dokumentace**
- **Dodržování standardů** (Coding style, Linters, Formattery (Prettier))

- **Vývojová prostředí**
  - **Obvykle se pracuje ve více prostředích**
    - Development (dev)
    - Staging - testovací prostředí, kopie produkce
    - Production (prod) - ostrá verze pro uživatele
- **CI/CD (Continuous Integration / Continuous Deployment)**
  - **Automatizace**
    - Spustí se testy, sestaví se aplikace, nasadí na server

Testování

- Ověření, že aplikace dělá to, co má
- Odhalení chy
- Kontrola výkonu, bezpečnosti, použitelnosti
- **Unit testy**
  - Testují nejmenší jednotku kódu
  - Obvykle jednu funkci nebo metodu
- **End-to-End testy (E2E)**
  - Testují celou aplikaci jako uživatel
- **Typy testování:**
  - **Zátěžové testy**
    - Jak se rychle načte stránka, kolik uživatelů najednou aplikaci zvládne
  - **Bezpečnostní testy**
    - Je aplikace odolná proti **SQL injection**, **XSS**, **CSRF**
    - Jsou hesla správně hashovaná
  - **Testy použitelnosti**
    - Zkušení uživatelé nebo cílová skupina zkouší aplikaci
    - Zjišťuje se, jestli je intuitivní

- **Kompatibilní testy**
  - **Cross-browser** - funguje v Chrome, Firefox, Safari, Edge?
  - **Cross-device** - funguje na mobilu, tabletu, PC?
  - **Responzivita** - zobrazuje se správně na různých velikostech obrazovky
- **Testy přístupnosti**
  - Funguje se screen readerem?
  - Je dodržen standart WCAG?

Nasazení

- Přeneseme aplikaci z vývojového prostředí na produkční server
- **Typy hostingu:**
  - **Sdílený hosting**
    - Levný jednoduchý
    - Sdílíme server s dalšími weby
  - **VPS**
    - Vlastní virtuální server s plnou kontrolou
  - **PaaS**
    - Nejjednodušší, nemusíme se starat o server
  - **IaaS**
- **Doména**
  - Registrace u doménové autority
  - Poplatek
- **SSL certifikát (HTTPS)**
  - Zajišťuje šifrovanou komunikaci mezi prohlížečem a serverem
    - Let's Encrypt

- **Postup**:
  1. Příprava produkční verze
  2. Nahrání na server
  3. Nastavení serveru
  4. Propojení domény a SSL
  5. Spuštění a ověření
- **Monitoring a logy:**
  - Monitoring dostupnosti
  - Error tracking
  - Analytika
  - Logy serveru

Údržba

- Zajišťuje, že web funguje, je bezpečný a reaguje na měnící se potřeby
- **Ce se tady dělá?:**
  - **Monitoring**
    - Dostupnost
    - Výkon
    - Chyby v aplikaci
    - Logy serveru
    - Analytika
  - **Aktualizace**
  - **Opravy chyb**
  - **Přidávání nových funkcí**
  - **Podpora uživatelů**
  - **SEO údržba**
  - **Správa obsahu** - u obsahových webů (nové články, produkty atd.)